

1 Normalization

1.1 Normal form violated by Booking receipt

Booking_receipt violates first normal form (1NF). The problem is the column seat_numbers: one cell can contain several values, for example "12A, 12B". In 1NF one cell must contain only one value. Therefore seat_numbers must be changed to seat_number, with one seat in each row.

1.2 Step by step normalization

Step 1 First normal form

Make seat_numbers atomic: write one seat number in one row. For example, instead of "12A, 12B" in one cell, create two rows: one row with 12A and one with 12B. The primary key can be booking_id, passenger_passport_number and flight_number together.

Booking_receipt_1NF(booking_id, passenger_passport_number, flight_number, passenger_full_name, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_number, ticket_price)

Step 2 Second normal form

Split the information that depends on only one part of the composite key. The passenger name belongs to the passport number, the ticket price belongs to booking_id, and flight information belongs to flight_number. Therefore these facts are moved into separate tables.

Booking_2NF(booking_id, ticket_price)

Passenger_2NF(passenger_passport_number, passenger_full_name)

Flight_2NF(flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name)

Booking_passenger_seat_2NF(booking_id, passenger_passport_number, flight_number, seat_number)

Step 3 Third normal form

Separate airport and airline information from the flight table. City belongs to an airport, not to a flight; airline name belongs to an airline, not to a flight. This removes unnecessary repetition. Underlined attributes are primary keys; FK labels are foreign keys.

Airline_3NF(airline_id, airline_name)

Airport_3NF(airport_id, airport_name, city)

Flight_3NF(flight_number, departure_airport_id [FK -> Airport_3NF.airport_id], arrival_airport_id [FK -> Airport_3NF.airport_id], airline_id [FK -> Airline_3NF.airline_id])

Passenger_3NF(passenger_passport_number, passenger_full_name)

Booking_3NF(booking_id, ticket_price)

Booking_passenger_seat_3NF has a composite primary key. The three underlined attributes below are both primary-key attributes and foreign keys.

Booking_passenger_seat_3NF(booking_id [FK -> Booking_3NF.booking_id], passenger_passport_number [FK -> Passenger_3NF.passenger_passport_number], flight_number [FK -> Flight_3NF.flight_number], seat_number)

1.3 Functional dependencies

These simple dependencies explain why the tables were split.

- booking_id -> ticket_price
- passenger_passport_number -> passenger_full_name
- flight_number -> departure_airport_name, arrival_airport_name, airline_name
- airport_name -> city
- airline_id -> airline_name
- airport_id -> airport_name, city
- (booking_id, passenger_passport_number, flight_number) -> seat_number

1.4 Check of Flights table for 3NF

The primary key of Flights is flight_id. The main functional dependency is flight_id -> sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at. The foreign-key identifiers departing_airport_id, arriving_airport_id and airline_id refer to facts stored in Airport and Airline, but the corresponding airport names, cities and airline properties are not duplicated in Flights. Therefore no transitive dependency between non-key attributes is present inside Flights, and the table is in 3NF on the stated design.

1.5 PostgreSQL implementation of the 3NF design and sample data

```
CREATE TABLE airline_n (  
    airline_id INT PRIMARY KEY,  
    airline_name VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE airport_n (  
    airport_id INT PRIMARY KEY,  
    airport_name VARCHAR(80) NOT NULL,  
    city VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE flight_n (  
    flight_number VARCHAR(10) PRIMARY KEY,  
    departure_airport_id INT NOT NULL,  
    arrival_airport_id INT NOT NULL,  
    airline_id INT NOT NULL,  
    FOREIGN KEY (departure_airport_id) REFERENCES airport_n(airport_id),  
    FOREIGN KEY (arrival_airport_id) REFERENCES airport_n(airport_id),  
    FOREIGN KEY (airline_id) REFERENCES airline_n(airline_id)  
);  
  
CREATE TABLE passenger_n (  
    passenger_passport_number VARCHAR(20) PRIMARY KEY,
```

```

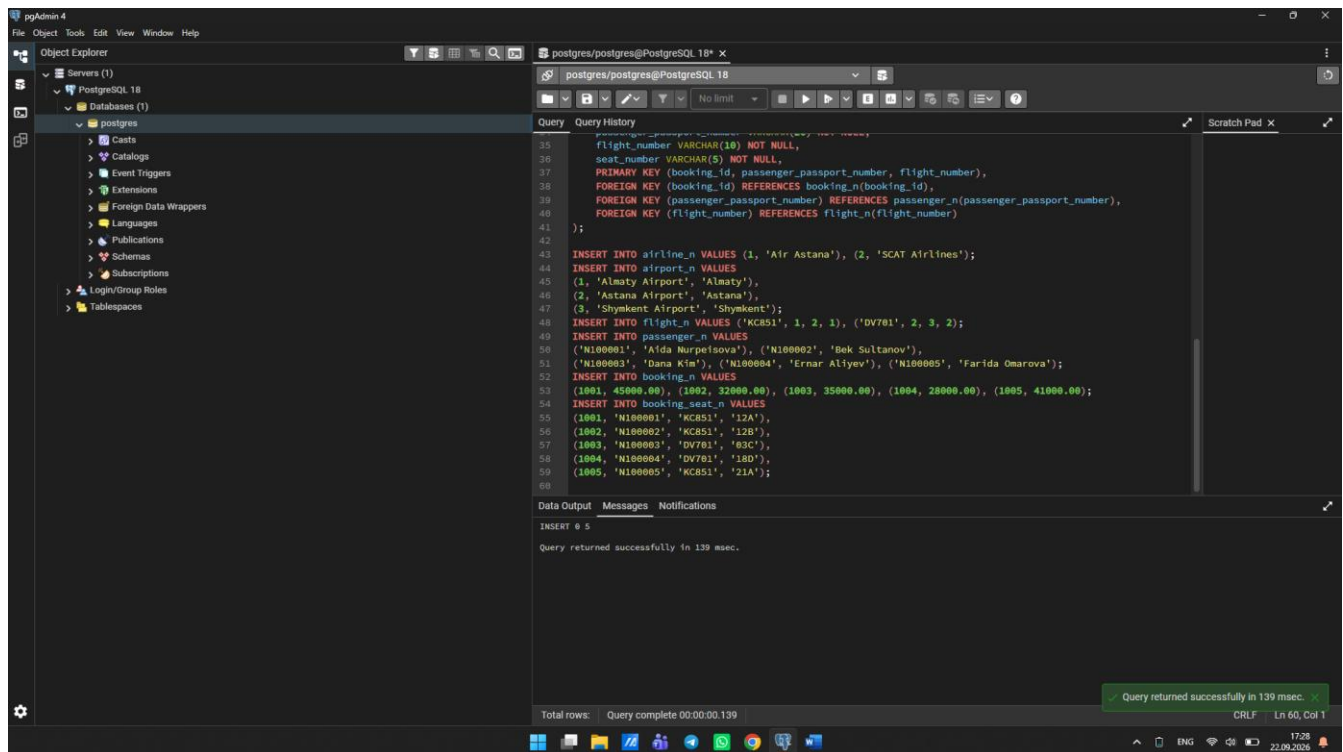
    passenger_full_name VARCHAR(100) NOT NULL
);

CREATE TABLE booking_n (
    booking_id INT PRIMARY KEY,
    ticket_price DECIMAL(7,2) NOT NULL
);

CREATE TABLE booking_seat_n (
    booking_id INT NOT NULL,
    passenger_passport_number VARCHAR(20) NOT NULL,
    flight_number VARCHAR(10) NOT NULL,
    seat_number VARCHAR(5) NOT NULL,
    PRIMARY KEY (booking_id, passenger_passport_number, flight_number),
    FOREIGN KEY (booking_id) REFERENCES booking_n(booking_id),
    FOREIGN KEY (passenger_passport_number) REFERENCES passenger_n(passenger_passport_number),
    FOREIGN KEY (flight_number) REFERENCES flight_n(flight_number)
);

INSERT INTO airline_n VALUES (1, 'Air Astana'), (2, 'SCAT Airlines');
INSERT INTO airport_n VALUES
(1, 'Almaty Airport', 'Almaty'),
(2, 'Astana Airport', 'Astana'),
(3, 'Shymkent Airport', 'Shymkent');
INSERT INTO flight_n VALUES ('KC851', 1, 2, 1), ('DV701', 2, 3, 2);
INSERT INTO passenger_n VALUES
('N100001', 'Aida Nurpeisova'), ('N100002', 'Bek Sultanov'),
('N100003', 'Dana Kim'), ('N100004', 'Ernar Aliyev'), ('N100005', 'Farida Omarova');
INSERT INTO booking_n VALUES
(1001, 45000.00), (1002, 32000.00), (1003, 35000.00), (1004, 28000.00), (1005, 41000.00);
INSERT INTO booking_seat_n VALUES
(1001, 'N100001', 'KC851', '12A'),
(1002, 'N100002', 'KC851', '12B'),
(1003, 'N100003', 'DV701', '03C'),
(1004, 'N100004', 'DV701', '18D'),
(1005, 'N100005', 'KC851', '21A');

```



pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (1)
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - Login/Group Roles
 - Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM airline_n;  
2 SELECT * FROM airport_n;  
3 SELECT * FROM flight_n;  
4 SELECT * FROM passenger_n;  
5 SELECT * FROM booking_seat_n;
```

Data Output Messages Notifications

airline_id [PK] integer	airline_name character varying (50)
1	Air Astana
2	SCAT Airlines

Showing rows: 1 to 2 Page No: 1 of 1

Successfully run. Total query runtime: 172 msec. 2 rows affected.

Total rows: 2 Query complete 00:00:00.172

CRLF Ln 1, Col 1

18:10 22.09.2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (1)
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - Login/Group Roles
 - Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM airline_n;  
2 SELECT * FROM airport_n;  
3 SELECT * FROM flight_n;  
4 SELECT * FROM passenger_n;  
5 SELECT * FROM booking_seat_n;
```

Data Output Messages Notifications

airport_id [PK] integer	airport_name character varying (50)	city character varying (50)
1	Almaty Airport	Almaty
2	Astana Airport	Astana
3	Shymkent Airport	Shymkent

Showing rows: 1 to 3 Page No: 1 of 1

Successfully run. Total query runtime: 163 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.163

CRLF Ln 2, Col 1

18:10 22.09.2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (1)
 - postgres
 - Castes
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - Login/Group Roles
 - Tablespaces

postgres/postgres@PostgreSQL 18

Query

```
1 SELECT * FROM airline_n;  
2 SELECT * FROM airport_n;  
3 SELECT * FROM flight_n;  
4 SELECT * FROM passenger_n;  
5 SELECT * FROM booking_seat_n;
```

Execute script

Data Output

	flight_number [PK] character varying (10)	departure_airport_id integer	arrival_airport_id integer	airline_id integer
1	KD851	1	2	1
2	DV701	2	3	2

Showing rows: 1 to 2 Page No: 1 of 1

Successfully run. Total query runtime: 211 msec. 2 rows affected.

Total rows: 2 Query complete 00:00:00.211

18:11 22.09.2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (1)
 - postgres
 - Castes
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - Login/Group Roles
 - Tablespaces

postgres/postgres@PostgreSQL 18

Query

```
1 SELECT * FROM airline_n;  
2 SELECT * FROM airport_n;  
3 SELECT * FROM flight_n;  
4 SELECT * FROM passenger_n;  
5 SELECT * FROM booking_seat_n;
```

Data Output

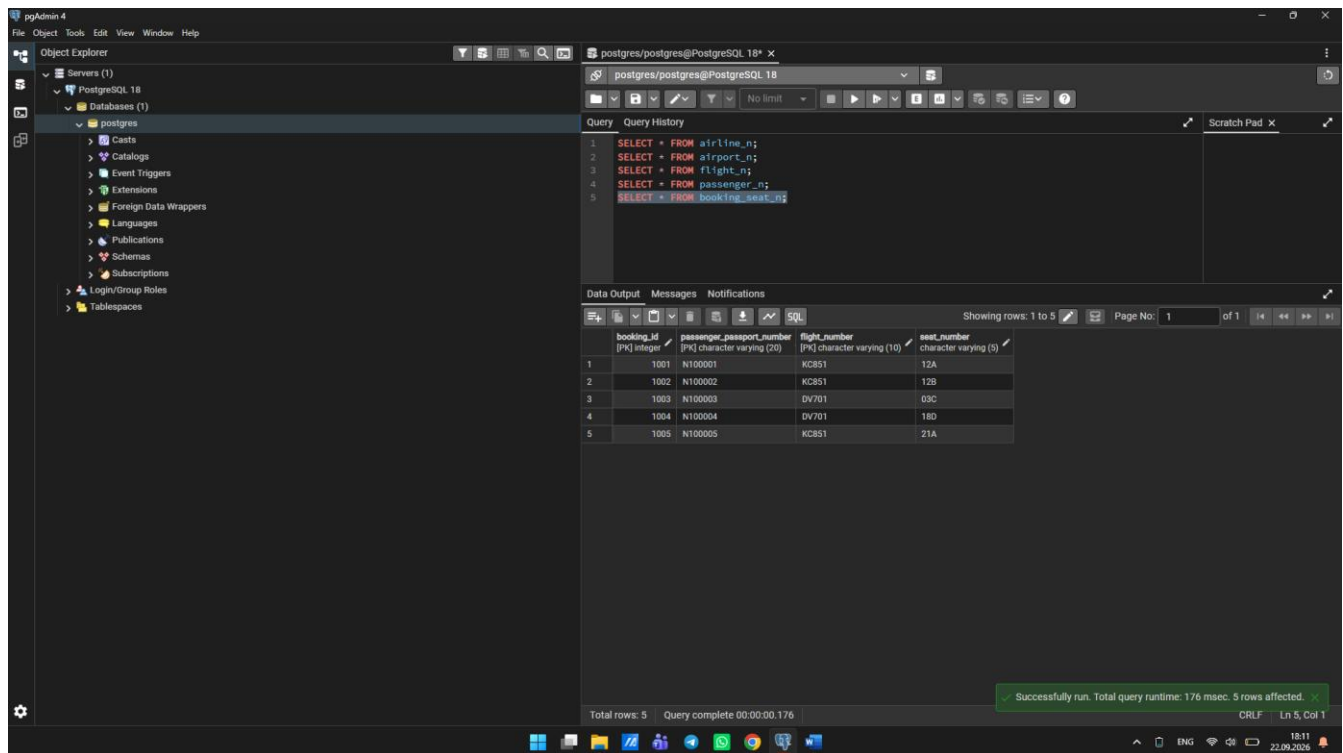
	passenger_passport_number [PK] character varying (20)	passenger_full_name character varying (100)
1	N100001	Aida Nurpesova
2	N100002	Bek Sultanov
3	N100003	Dana Kim
4	N100004	Emar Aliyev
5	N100005	Farida Omarova

Showing rows: 1 to 5 Page No: 1 of 1

Successfully run. Total query runtime: 171 msec. 5 rows affected.

Total rows: 5 Query complete 00:00:00.171

18:11 22.09.2026



2 DDL for the airport database

2.1 Review of the submitted CREATE TABLE statements

The submitted CREATE TABLE statements are correct for the initial DDL requirement: each requested table and column is present, the data types match the assignment, all attributes are NOT NULL, and each table has a primary key. PRIMARY KEY already implies NOT NULL, so writing both on the ID column is redundant but valid. Foreign keys are intentionally added after the requested rename and column changes in the next script.

2.2 Initial table creation

```
DROP TABLE IF EXISTS baggage_check, baggage, boarding_pass, booking_flight,
    booking, flights, security_check, passengers, airport, airline_info CASCADE;

CREATE TABLE airline_info (
    airline_id INT PRIMARY KEY NOT NULL,
    airline_code VARCHAR(30) NOT NULL,
    airline_name VARCHAR(50) NOT NULL,
    airline_country VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    info VARCHAR(50) NOT NULL
);

CREATE TABLE airport (
    airport_id INT PRIMARY KEY NOT NULL,
    airport_name VARCHAR(50) NOT NULL,
    country VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL,
```

```

        created_at TIMESTAMP NOT NULL,
        updated_at TIMESTAMP NOT NULL
    );

CREATE TABLE passengers (
    passenger_id INT PRIMARY KEY NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(50) NOT NULL,
    country_of_citizenship VARCHAR(50) NOT NULL,
    country_of_residence VARCHAR(50) NOT NULL,
    passport_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE flights (
    flight_id INT PRIMARY KEY NOT NULL,
    sch_departure_time TIMESTAMP NOT NULL,
    sch_arrival_time TIMESTAMP NOT NULL,
    departing_airport_id INT NOT NULL,
    arriving_airport_id INT NOT NULL,
    departing_gate VARCHAR(50) NOT NULL,
    arriving_gate VARCHAR(50) NOT NULL,
    airline_id INT NOT NULL,
    act_departure_time TIMESTAMP NOT NULL,
    act_arrival_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE booking (
    booking_id INT PRIMARY KEY NOT NULL,
    flight_id INT NOT NULL,
    passenger_id INT NOT NULL,
    booking_platform VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    price DECIMAL(7,2) NOT NULL
);

CREATE TABLE baggage_check (
    baggage_check_id INT PRIMARY KEY NOT NULL,
    check_result VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INT NOT NULL,
    passenger_id INT NOT NULL
);

CREATE TABLE baggage (
    baggage_id INT PRIMARY KEY NOT NULL,
    weight_in_kg DECIMAL(4,2) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INT NOT NULL
);

CREATE TABLE boarding_pass (
    boarding_pass_id INT PRIMARY KEY NOT NULL,
    booking_id INT NOT NULL,
    seat VARCHAR(50) NOT NULL,
    boarding_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE booking_flight (
    booking_flight_id INT PRIMARY KEY NOT NULL,
    booking_id INT NOT NULL,
    flight_id INT NOT NULL,
    created_at TIMESTAMP NOT NULL,

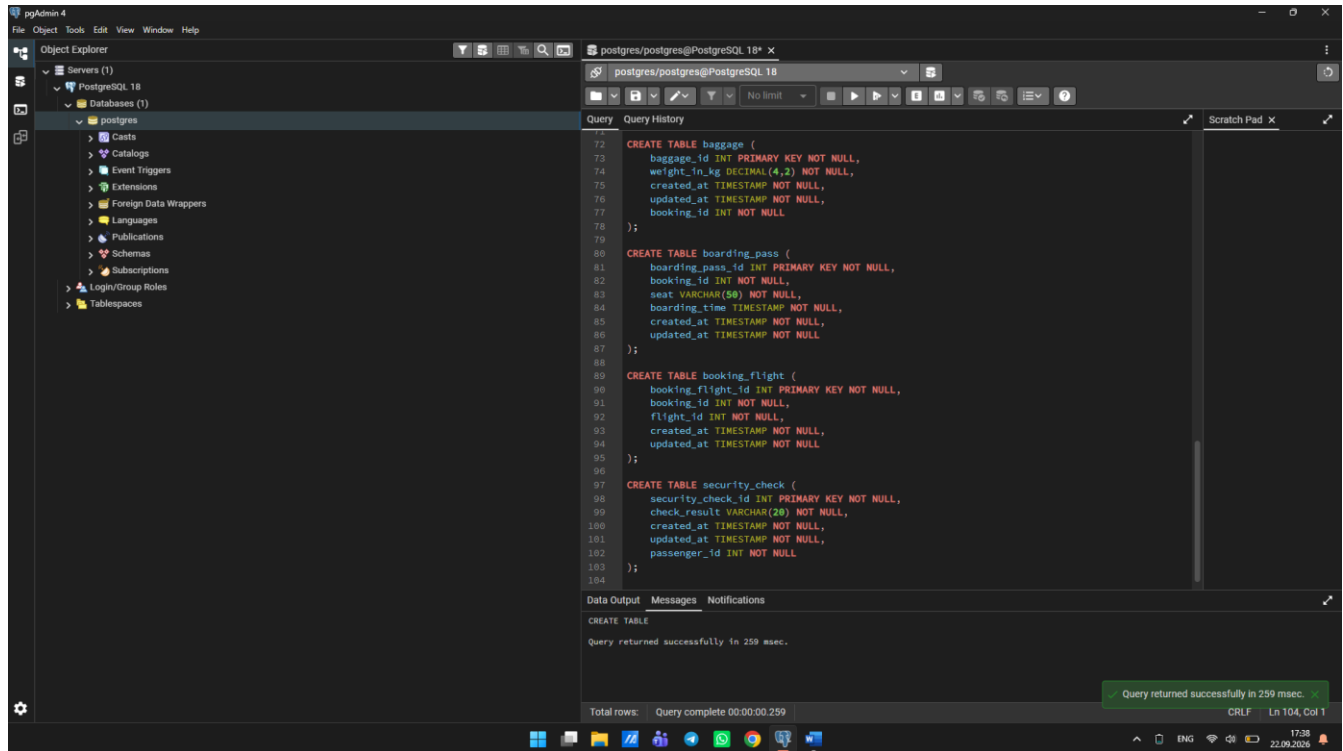
```

```

        updated_at TIMESTAMP NOT NULL
    );

CREATE TABLE security_check (
    security_check_id INT PRIMARY KEY NOT NULL,
    check_result VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    passenger_id INT NOT NULL
);

```



2.3 Required changes and relationships

```

ALTER TABLE airline_info RENAME TO airline;
ALTER TABLE booking RENAME COLUMN price TO ticket_price;
ALTER TABLE flights ALTER COLUMN departing_gate TYPE TEXT;
ALTER TABLE airline DROP COLUMN info;

ALTER TABLE security_check
    ADD CONSTRAINT fk_security_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id);

ALTER TABLE booking
    ADD CONSTRAINT fk_booking_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id);

ALTER TABLE baggage_check
    ADD CONSTRAINT fk_baggage_check_booking
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id),
    ADD CONSTRAINT fk_baggage_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id);

ALTER TABLE baggage
    ADD CONSTRAINT fk_baggage_booking
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id);

ALTER TABLE boarding_pass
    ADD CONSTRAINT fk_boarding_pass_booking

```



```
FOREIGN KEY (booking_id) REFERENCES booking(booking_id);

ALTER TABLE booking_flight
  ADD CONSTRAINT fk_booking_flight_booking
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id),
  ADD CONSTRAINT fk_booking_flight_flight
  FOREIGN KEY (flight_id) REFERENCES flights(flight_id);

ALTER TABLE flights
  ADD CONSTRAINT fk_flights_departing_airport
  FOREIGN KEY (departing_airport_id) REFERENCES airport(airport_id),
  ADD CONSTRAINT fk_flights_arriving_airport
  FOREIGN KEY (arriving_airport_id) REFERENCES airport(airport_id),
  ADD CONSTRAINT fk_flights_airline
  FOREIGN KEY (airline_id) REFERENCES airline(airline_id);
```

